

PART I. INTERACTIVE BEHAVIOR

1. Interaction vs. animation in use of Behavior class

- animation: in response to the passing of time
- interaction: in response to user activities

2. Input devices for object manipulation

- Java3D has access to keyboards/mice via Java API
- Implementing `InputDevice` interface for other devices

3. Activities of interactive behavior

- Essential to interactivity, object can be selected by
 - “hard-wiring” with mouse or key events
 - picking with mouse buttons at run time

- Interactively manipulating subjects with input devices
 - Keyboard-based interaction
 - Use of `KeyNavigatorBehavior` utility class
 - view navigation with special keys, e.g., arrow keys
 - self-defined behavior for object manipulation
 - Use of `KeyListener` interface
 - overriding key events for self-defined behaviors
 - Mouse-based interaction
 - Use of abstract `MouseBehavior` class
 - subclasses for rotation, translation, and zooming
 - changed behavior by overriding stimuli process
 - Use of `MouseListener` interface
 - overriding mouse events with `PickTool`

4. Keyboard-based interaction

- The keyboard is the simplest hardware for interaction with a 3D virtual world.
 - Navigation with KeyNavigatorBehavior utility class
 - Special-purpose, self-defined behavior of individual keys
- Use of KeyNavigatorBehavior utility class
 - An example of default behavior for view navigation
 - four arrow keys, plus up(PgUp)/down(PgDn)/reset(=)

```
private KeyNavigatorBehavior keyNavigation(SimpleUniverse simple_U) {
    ViewingPlatform view_platfm = simple_U.getViewingPlatform();
    TransformGroup view_TG = view_platfm.getViewPlatformTransform();
    KeyNavigatorBehavior keyNavBeh = new KeyNavigatorBehavior(view_TG);
    BoundingSphere view_bounds = new BoundingSphere(new Point3d(), 20.0);
    keyNavBeh.setSchedulingBounds(view_bounds);
    return keyNavBeh;
}
```

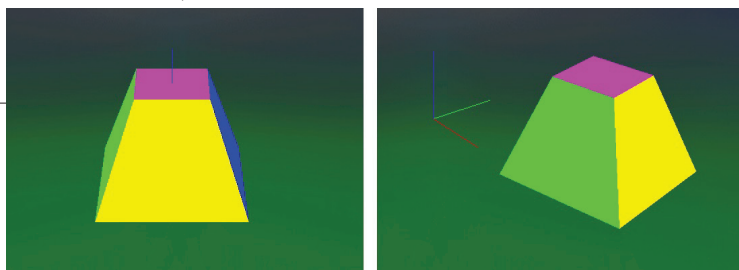
- An example of modified behavior for object manipulation

```
public BranchGroup createSceneGraph() {
    BranchGroup scene = new BranchGroup();

    TransformGroup sceneTG = new TransformGroup();
    sceneTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    scene.addChild(sceneTG);
    sceneTG.addChild(ShapeIndexedQuad.indexedQuadShape(1.0f));

    // Implement behavior listeners for shape and Camera
    KeyNavigatorBehavior myRotationbehavior = new KeyNavigatorBehavior(sceneTG);
    BehaviorArrowKey myViewRotationbehavior = new BehaviorArrowKey(ourView, sceneTG);
    myRotationbehavior.setSchedulingBounds(new BoundingSphere());
    scene.addChild(myRotationbehavior);
    myViewRotationbehavior.setSchedulingBounds(new BoundingSphere());
    scene.addChild(myViewRotationbehavior);

    return scene;
}
```



```

public class BehaviorArrowKey extends Behavior {
    private TransformGroup chaseTG;
    private WakeupOnAWTEvent wEnter;

    public BehaviorArrowKey(ViewingPlatform targetVP, TransformGroup chasedTG) {
        chaseTG = chasedTG;
        targetVP.getViewPlatformTransform();
    }
    public void initialize() {
        wEnter = new WakeupOnAWTEvent(KeyEvent.KEY_PRESSED);
        wakeupOn(wEnter); // decide when behavior becomes live
    }
    public void processStimulus(Iterator<WakeupCriterion> criteria) {
        Transform3D newrot = new Transform3D();
        chaseTG.getTransform(newrot); // process when arrow keys pressed,
        Vector3d vct = new Vector3d(); // including up|down|left|right
        newrot.get(vct); vct.y += 3d; vct.z -= 3d;

        wakeupOn(wEnter); // decide when behavior becomes live
    }
}

```

• Key events as an implementation of KeyListener

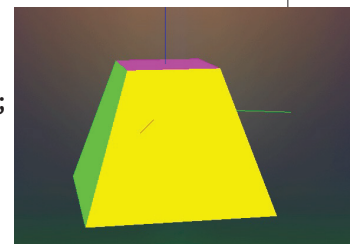
```

public void keyPressed(KeyEvent evt) {
    Transform3D tmp = new Transform3D();
    if ((evt.getKeyCode() == KeyEvent.VK_R)) {
        Matrix4d mat = new Matrix4d();
        rotationBG.getTransform(tmp); // retrieve from working BG
        tmp.get(mat); // save current matrix to 'mat'
        Matrix4d m4d = new Matrix4d(); // define an identity matrix
        m4d.rotY(0.1); mat.mul(m4d); // define and perform rotation
        tmp.set(mat); rotationBG.setTransform(tmp); // set back to the working BG
    }
    if (evt.getKeyCode() == KeyEvent.VK_O) {
        tmp = new Transform3D(); // the same matrix as L.46
        rotationBG.setTransform(tmp); // reset to the original
    }
}

public BranchGroup createSceneGraph() {
    Transform3D trans_cube = new Transform3D();
    rotationBG = new TransformGroup(trans_cube);
    rotationBG.setCapability(TransformGroup.ALLOW_TRANSFORM_READ);
    rotationBG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    rotationBG.addChild(ShapeIndexedQuad.indexedQuadShape(0.5f));

    BranchGroup contentBG = new BranchGroup();
    contentBG.addChild(rotationBG);
    return contentBG;
}

```



5. Mouse-based object manipulation

- Three pre-defined types of mouse dragging actions
 - **MouseRotate**: left mouse button for rotation
 - **MouseTranslate**: right mouse button for translation
 - **MouseZoom**: middle mouse button for zooming

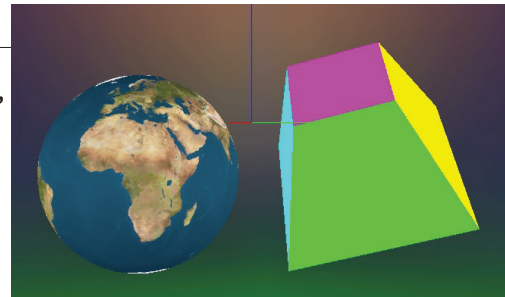
MouseBehavior Class	Response to Mouse Action	Mouse Action
MouseRotate	Rotate object	Left-button held with mouse movement
MouseTranslate	Translate object in a plane parallel to image plate	Right-button held with mouse movement
MouseZoom	Translate the object in a plane orthogonal to the image plate	Middle-button held with mouse movement

```

public BranchGroup createSceneGraph(Canvas3D c) {
    Vector3f[] vctr = {new Vector3f(-0.5f, 0.0f, 0.5f),
                      new Vector3f(0.5f, 0.0f, -0.5f)};
    Transform3D transfm = new Transform3D();
    TransformGroup transfmTG;
    BranchGroup scene = new BranchGroup();
    for (int i = 0; i < 2; i++) {
        transfm.setTranslation(vctr[i]);
        transfmTG = new TransformGroup(transfm);
        if (i < 1)
            transfmTG.addChild(AppearanceTexture2D.textureSphere(0.35f, 60));
        else
            transfmTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.9f));
        transfmTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
        transfmTG.setCapability(TransformGroup.ALLOW_TRANSFORM_READ);
        transfmTG.setCapability(TransformGroup.ENABLE_PICK_REPORTING);
        scene.addChild(transfmTG);
    }
    Behavior pickBeh[] = new Behavior[3];
    pickBeh[0] = new PickTranslateBehavior(scene, c, CommonsXY.boundT);
    pickBeh[1] = new PickZoomBehavior(scene, c, CommonsXY.boundT);
    pickBeh[2] = new PickRotateBehavior(scene, c, CommonsXY.boundT);

    boolean geo_tag = true; // true false
    int mode = (geo_tag ? PickInfo.PICK_GEOMETRY : PickInfo.PICK_BOUNDS);
    for (int i = 0; i < 3; i++) {
        ((PickMouseBehavior) pickBeh[i]).setMode(mode);
        scene.addChild(pickBeh[i]);
    }
    return scene;
}

```

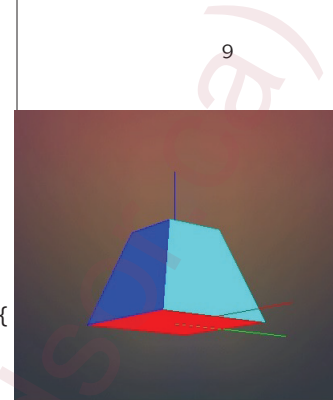


```

public class MyMouseMotion extends Behavior {
    private TransformGroup targetTG;
    private Transform3D trans = new Transform3D();
    private float transY = 0.0f;
    private WakeupOnAWTEvent wEnter;

    public MyMouseMotion(TransformGroup targetTG) {
        this.targetTG = targetTG;
    }
    public void initialize() { // set initial wakeup condition;
        wEnter = new WakeupOnAWTEvent(MouseEvent.MOUSE_PRESSED);
        wakeupOn(wEnter); // called when behavior becomes live
        // decode event ; do what is necessary
    }
    public void processStimulus(Iterator<WakeupCriterion> criteria) {
        if (++transY > 6.0f)
            transY = -2.0f;
        trans.set(new Vector3f(0.0f, transY / 10.0f, 0.0f));
        targetTG.setTransform(trans);
        wakeupOn(wEnter);
    }
}

```



- Modified behavior with self-defined mouse actions

```

private static void createContent(BranchGroup scene) {
    Transform3D translate = new Transform3D();
    translate.set(new Vector3f(0.0f, -0.3f, 0.0f));
    TransformGroup transShape = new TransformGroup(translate);

    TransformGroup moveTG = new TransformGroup();
    moveTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    moveTG.setCapability(TransformGroup.ALLOW_TRANSFORM_READ);

    MyMouseMotion myMovementBehavior = new MyMouseMotion(moveTG);
    myMovementBehavior.setSchedulingBounds(new BoundingSphere());
    transShape.addChild(moveTG);
    moveTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.4f));
    moveTG.addChild(myMovementBehavior);

    scene.addChild(transShape);
}

```

- Use of MouseListener interface
 - The picking API divides into two portions
 - Control: click with a 2D mouse (or move a wand)
 - Selection: find shapes meeting the search criteria
 - Using the mouse for a pick needs PickTool to override mouse events for self-defined behaviors
 - Construct a PickRay from the eye passing through the 2D mouse screen point
 - Initiate a pick to find all pick hits along the ray, sorted from closest to furthest
 - Get the first pick hit in the returned data
 - Do something to that picked shape
 - (Re)declare interest in mouse events

```

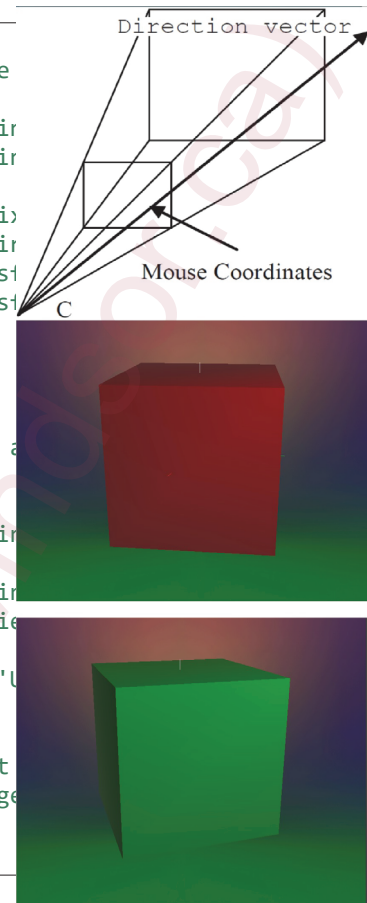
public void mouseClicked(MouseEvent event) {
    int x = event.getX(); int y = event.getY();           // mouse
    Point3d point3d = new Point3d(), center = new Point3d();
    canvas.getPixelLocationInImagePlate(x, y, point3d); // obtain
    canvas.getCenterEyeInImagePlate(center);             // obtain

    Transform3D transform3D = new Transform3D();         // matrix
    canvas.getImagePlateToVworld(transform3D);           // to Vir
    transform3D.transform(point3d);                     // trans
    transform3D.transform(center);                       // trans

    Vector3d mouseVec = new Vector3d();
    mouseVec.sub(point3d, center);
    mouseVec.normalize();
    pickTool.setShapeRay(point3d, mouseVec);            // send a

    if (pickTool.pickClosest() != null) {
        PickResult pickResult = pickTool.pickClosest(); // obtain
        Box box = (Box)pickResult.getNode(PickResult.PRIMITIVE);
        Appearance app = new Appearance();              // origin
        if ((int) box.getUserData() == 0) {              // retrie
            app.setMaterial(setMaterial(commons.Green));
            box.setUserData(1); }                       // set 'U
        else {
            app.setMaterial(setMaterial(commons.Red));
            box.setUserData(0); }                       // reset
        box.setAppearance(app);                         // change
    }
}

```



Java3D...Interaction

12

```

private static void oneBox(BranchGroup scene) {
    Appearance app = new Appearance();
    app.setMaterial(setMaterial(commons.Red));           // create a box with
    Box box = new Box(1.1f, 1.1f, 1.1f, Primitive.GENERATE_NORMALS |
        Primitive.ENABLE_APPEARANCE_MODIFY, app);
    box.setName("box");                                  // 'Name' is retriev
    box.setUserData(0);                                  // 'UserData' retriev

    TransformGroup objTG = new TransformGroup();
    objTG.addChild(box);                                  // add 'box' to the
    Alpha rotationAlpha = new Alpha(-1, 10000);
    scene.addChild(rotateBehavior(objTG, rotationAlpha)); // make 'objTG' cont

    scene.addChild(objTG);
}

public static BranchGroup createScene() {
    BranchGroup sceneBG = new BranchGroup();
    oneBox(sceneBG);
    pickTool = new PickTool( sceneBG );                 // allow picking of
    pickTool.setMode(PickTool.BOUNDS);
    sceneBG.compile();                                    // optimize 'sceneBG'
    return sceneBG;
}

public InteractionPickRay(BranchGroup sceneBG) {
    GraphicsConfiguration config = SimpleUniverse.getPreferredConfiguration();
    canvas = new Canvas3D(config);
    canvas.addMouseListener(this);                     // NOTE: enable mouse

    SimpleUniverse su = new SimpleUniverse(canvas);     // create a SimpleUn
}

```


PART II. INTERACTION WITH SOUNDS

1. Two types of sound to add to a virtual environment

- Localized sounds – sounds with a position
 - User interface sounds, e.g., clicks, alerts
 - Data sonification and game sounds, e.g., laser blasters
- Background sounds – sounds filling an environment
 - Presentation sounds, e.g., voice over, narration
 - Environment/background sounds, e.g., ocean waves

2. Elements of sound

- Sound data to play, initial gain (overall volume), looping parameters, and playback priority
- Scheduling bounds (like a behavior)

- Sound is a Leaf node
 - Plays *sound data* describing a digital waveform
 - Data loaded by a `MediaContainer` from a file on disk or on the Web
 - The `MediaContainer` class provides functionality to load sound files given a URL or file path
 - Typical sound file formats include:
 - AIF: standard cross-platform format
 - AU: standard Sun format
 - WAV: standard PC format
 - Default values
 - disabled, having a gain of 1.0, and not looped

3. Using the extended background and PointSound

- The waves of BackgroundSound come from all directions
 - It floods an environment at constant volume, similar idea as an AmbientLight
 - Use background sounds for presentation sound, environment sounds, or background music
- The waves of PointSound emit radially from a point in all directions, similar idea as a PointLight
 - Use point sounds to simulate local sounds, such as interface sounds (e.g., clicks), data sonification, and game sounds (e.g., laser blasters)
- May have multiple of both types of sounds playing

◦ An example of BackgroundSound and PointSound

```
private void enableAudio(SimpleUniverse simple_U) {
    JOALMixer mixer = null; // create a joalmixer
    Viewer viewer = simple_U.getViewer();
    viewer.getView().setBackClipDistance(20.0f); // disappear beyond 20f

    if (mixer == null && viewer.getView().getUserHeadToWorldEnable()) {
        mixer = new JOALMixer(viewer.getPhysicalEnvironment());
        if (!mixer.initialize()) { // add audio device
            System.out.println("Open AL failed to init");
            viewer.getPhysicalEnvironment().setAudioDevice(null);
        }
    }

    private BackgroundSound bkgdSound() {
        BackgroundSound bgs = new BackgroundSound(); // create a background sound
        bgs.setInitialGain( 0.05f ); // lower its volume
        BackgroundSoundBehavior player = // create the sound behavior
            new BackgroundSoundBehavior( bgs, locateSound("ocean"));
        player.setSchedulingBounds(CommonsXY.boundH); // set scheduling bound

        return bgs;
    }
}
```



```

private URL locateSound(String fn) {
    URL url = null;
    String filename = "sounds/" + fn + ".wav";           // specify file location
    try {                                                 // locate the file
        url = new URL("file", "localhost", filename);
    } catch (Exception e) {
        System.out.println("Can't open " + filename);
    }
    return url;
}
private PointSound pointSound() {
    MediaContainer pointMedia = new MediaContainer(locateSound("magic_bells"));
    pointMedia.setCacheEnable(true);                    // enable cache
    PointSound pointSound = new PointSound();           // create point sound
    pointSound.setSoundData(pointMedia);                // set sound file
    pointSound.setEnable(true);                          // enable sound
    pointSound.setInitialGain(1.0f);                     // set initial gain
    pointSound.setLoop(-1);                              // 0: once; -1: loop
    pointSound.setSchedulingBounds(CommonsXY.boundH);    // set schuduling
    return pointSound;
}

```

4. Using **OpenAL** and **JOAL**

- **OpenAL** (Open Audio Library) is a cross-platform audio application programming interface (API).
 - It is designed for efficient rendering of multichannel three-dimensional positional audio.
 - Its main entities include:
 - (Multi-)buffers to store sound clips from WAV files
 - Sources to tell where each of the samples is played
 - A single listener, his/her position and velocity
- **JOAL** is a wrapper library to access **OpenAL** via Java
 - It allows Java programmers to use 3D sound in their applications.
 - Its 2D/3D audio features are really those of **OpenAL**

```

public class SoundListenerJOAL {
    // use hash map to store for sounds
    private HashMap<String, int[]> buffersMap; // name-buffer pairs
    private HashMap<String, int[]> sourcesMap; // name-source pairs
    private final static String SOUND_DIR = "sounds/"; // folder of WAV files
    // specify listener's info
    private float xLis, yLis, zLis; // current position
    private float[] oriLis; // orientation
    private int angleLis = 0; // rotation along z-axis

    private AL al; // to access the JOAL API

    public static void main(String[] args) {
        String soundName = "cow"; // filename: "cow"/"ocean"
        SoundListenerJOAL soundMan = // link to OpenAL &
            new SoundListenerJOAL(); // position the listener
        soundMan.moveListener(1, 0); // listener at (1,0,0) now

        if (!soundMan.load(soundName, 2, 0, 0, true)) // load the sound at (2,0,0)
            System.exit(1);
        soundMan.play(soundName); // play the sound

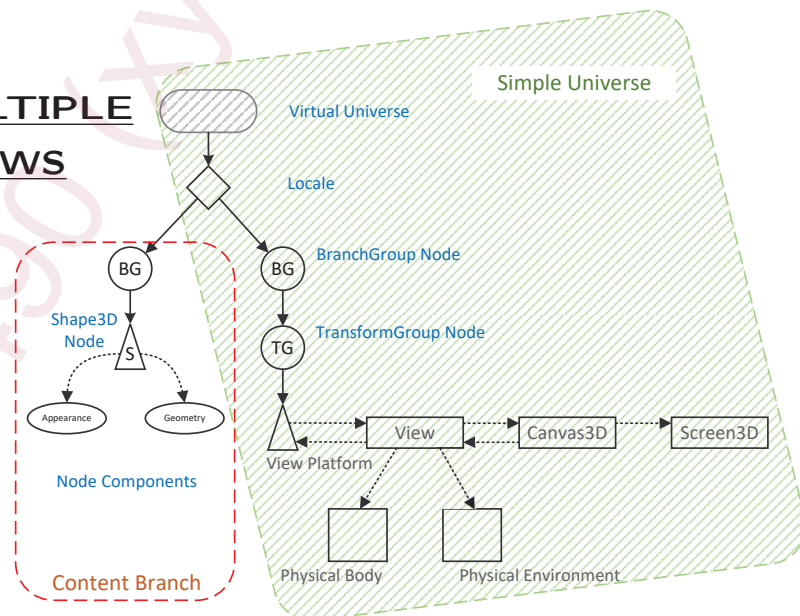
        float x = -31f;
        for (int i = 0; i < 60; i++) {
            // soundMan.turnListener(6); // rotate listener anti-clockwise 6 degrees
            soundMan.setPos(soundName, ++x / 10, 0, 0);
            try {
                Thread.sleep(300); // sleep for 0.3 secs
            } catch (InterruptedException ex) {
            }
        }
        soundMan.cleanUp(); // remove the hash maps
    }
}

```

Java3D...Interaction

PART III. MULTIPLE VIEWS

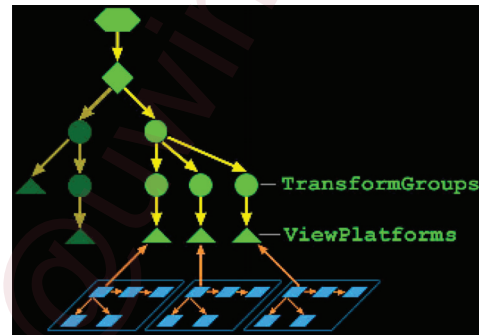
1. Another look at the view branch



2. A **ViewPlatform** defines a view point within the scene

- It defines a frame of reference for the user's position and orientation in the virtual world

- A View represents the user on a ViewPlatform
 - It manages the rendering of the scene into a screen region from the user's viewpoint
 - That screen region is a Canvas3D (extends AWT Canvas)
 - Typically, add a Canvas3D to a Java Frame, then point a View at that canvas
- A scene graph may contain multiple ViewPlatforms
 - When a View is attached to a platform, the scene is rendered from that viewpoint
 - An example of different view points



- “Teleports” the user to one to another viewpoint



```
public void actionPerformed( ActionEvent event ) {
    KeyNavigatorBehavior key = (KeyNavigatorBehavior)m_KeyHashtable.get(event.getActionCommand());
    Object[] keysArray = m_KeyHashtable.values( ).toArray( );
    for( int n = 0; n < keysArray.length; n++ ) {
        KeyNavigatorBehavior keyAtIndex = (KeyNavigatorBehavior) keysArray[n];
        keyAtIndex.setEnabled( keyAtIndex == key );
        if( keyAtIndex == key )
            System.out.println( "Enabled: " + event.getActionCommand( ) );
    }
}
```

```

public AppMultiPlatforms( ) {
    m_KeyHashtable = new Hashtable<String, KeyNavigatorBehavior>( );

    Canvas3D[] canvas3D = new Canvas3D[3];
    GraphicsConfiguration config = SimpleUniverse.getPreferredConfiguration( );
    for (int i = 0; i < 3; i++) {
        canvas3D[i] = new Canvas3D( config );
        canvas3D[i].setSize( width, height );
        add( canvas3D[i] );
    }
    ViewingPlatform vp = new ViewingPlatform(2);           // a VP with 2 TG about it
    Viewer viewer = new Viewer( canvas3D[0] );           // 1st Viewer attached c3D[0]
    Transform3D t3d = new Transform3D( );
    t3d.rotX( Math.PI / 2.0 );                           // rotate and position the 1st ~
    t3d.setTranslation( new Vector3d( 0, 0, -10 ) );     // viewer looking down from top
    t3d.invert( );
    MultiTransformGroup mtg = vp.getMultiTransformGroup( );
    mtg.getTransformGroup(0).setTransform( t3d );

    SimpleUniverse su = new SimpleUniverse(vp, viewer); // a SU with one Vp and 3 Viewers
    Locale lcl = su.getLocale();
    lcl.addBranchGraph( createViewer( canvas3D[1], "F-L", CommonsXY.Orange, -3, 1, 3 ) );
    lcl.addBranchGraph( createViewer( canvas3D[2], "B-R", CommonsXY.Cyan, 3, 1, -3 ) );

    BranchGroup scene = sidedCube( );                   // create a one-cube scene
    scene.addChild( CommonsXY.threeAxes( CommonsXY.Blue, 2f ) );
    TransformGroup scene_TG = new TransformGroup();
    AppearanceExtra.addLights( scene_TG, CommonsXY.Orange );
    scene.addChild( LightFog.createBkground( CommonsXY.White, CommonsXY.boundH ) );
    scene.addChild( scene_TG );

    scene.compile();
    su.addBranchGraph( scene );
}

```

```

ViewingPlatform createViewer(Canvas3D canvas3D, String name, Color3f clr,
    double x, double y, double z) {
    // a Canvas3D can only be attached to a single Viewer
    Viewer viewer = new Viewer( canvas3D );           // attach a Viewer to its canvas
    ViewingPlatform vp = new ViewingPlatform( 1 );     // 1 VP with 1 TG above
                                                    // assign PG to the Viewer
    vp.setPlatformGeometry( labelPlatformGeometry( name ) );
    viewer.setAvatar( createViewerAvatar( name, clr ) ); // assign VA to the Viewer

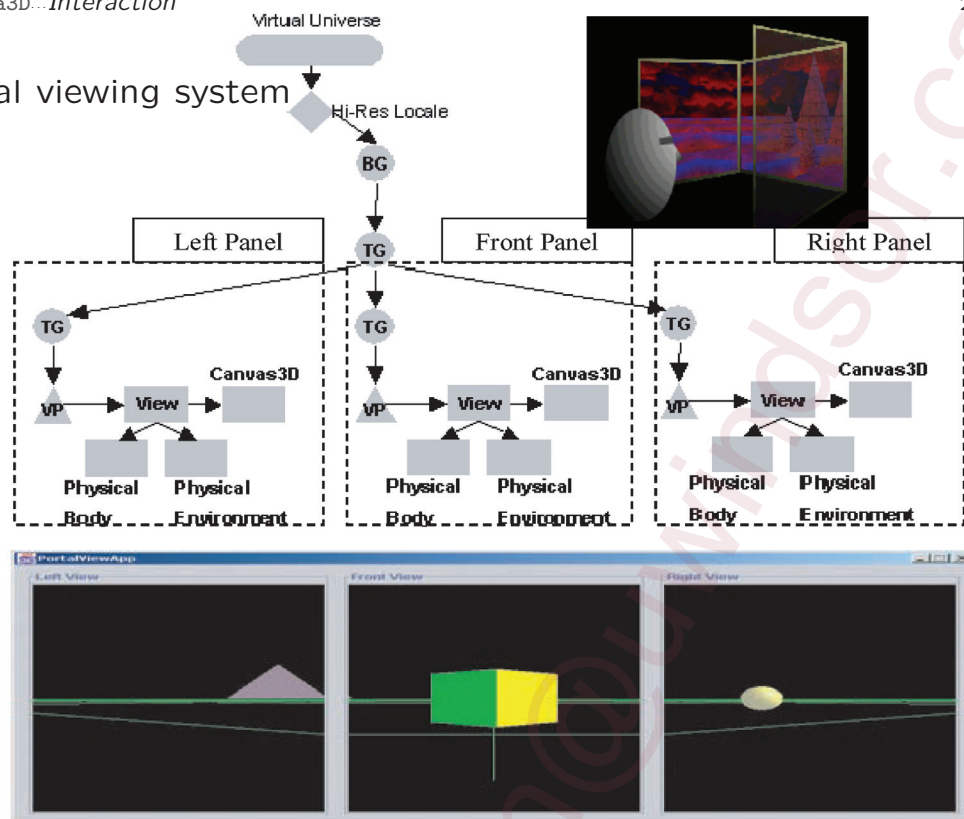
    Point3d center = new Point3d(0, 0, 0);           // define where the eye looks at
    Vector3d up = new Vector3d(0, 1, 0);             // define camera's up direction
    Transform3D viewTM = new Transform3D();
    Point3d eye = new Point3d(x, y, z);              // define eye's location
    viewTM.lookAt(eye, center, up);
    viewTM.invert();
    vp.getViewPlatformTransform().setTransform(viewTM); // set VP with 'viewTG'

    // set TG's capabilities to allow KeyNavigatorBehavior modify the Viewer's position
    vp.getViewPlatformTransform().setCapability( TransformGroup.ALLOW_TRANSFORM_WRITE );
    vp.getViewPlatformTransform().setCapability( TransformGroup.ALLOW_TRANSFORM_READ );
    KeyNavigatorBehavior key = new KeyNavigatorBehavior( vp.getViewPlatformTransform( ) );
    key.setSchedulingBounds( CommonsXY.boundT );      // enable viewer navigation
    key.setEnable( false );
    vp.addChild( key );                               // add KeyNavigatorBehavior to VP
    viewer.setViewingPlatform( vp );                 // set VP for the Viewer
    m_KeyHashtable.put( name, key );                 // label the Viewer
    Button button = new Button( name );
    button.addActionListener( this );                // button to switch the Viewer ON
    add( button );

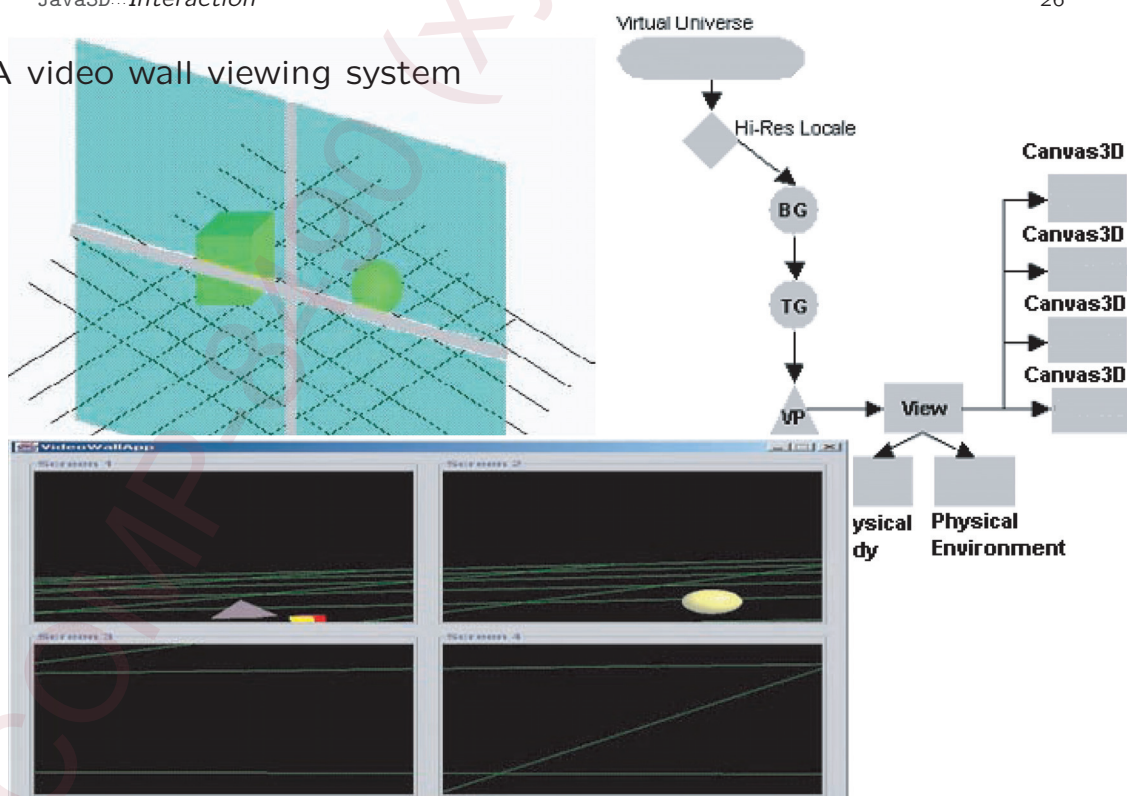
    return vp;
}

```

- A portal viewing system



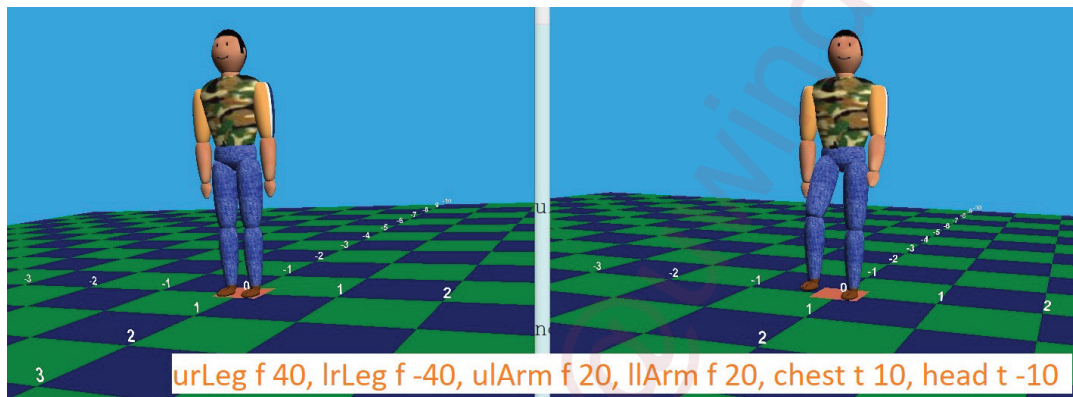
- A video wall viewing system



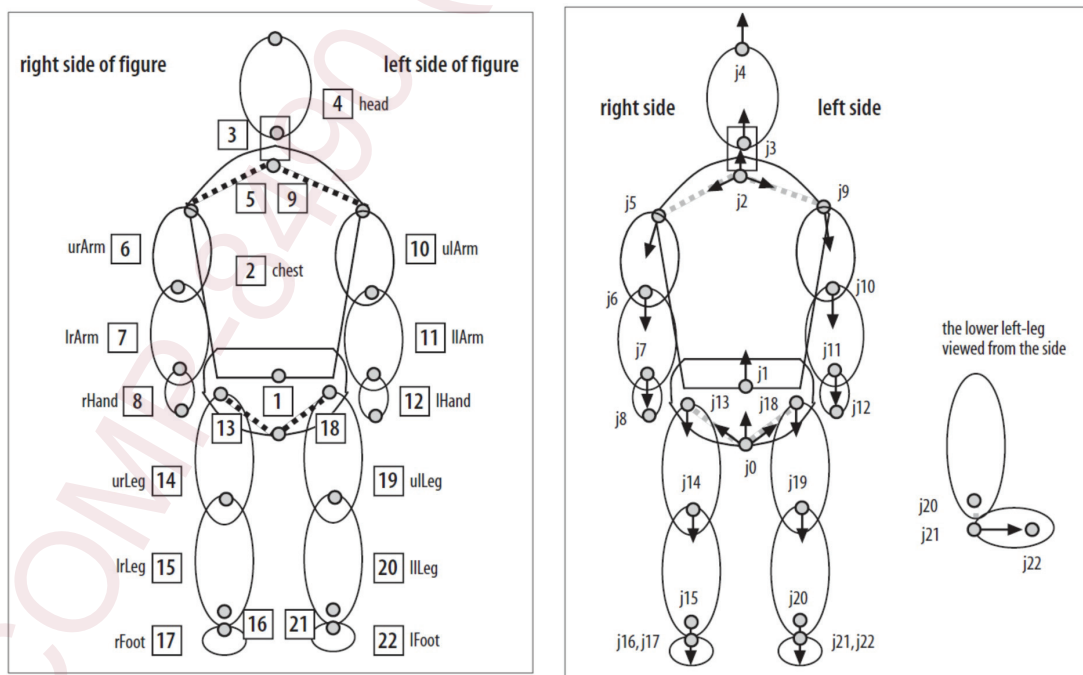
PART IV. AN ARTICULATED FIGURE

1. An articulated figure is a human-like structure composed of movable and rotatable limbs, which can be instructed to walk, run, and jump around a checkerboard floor.

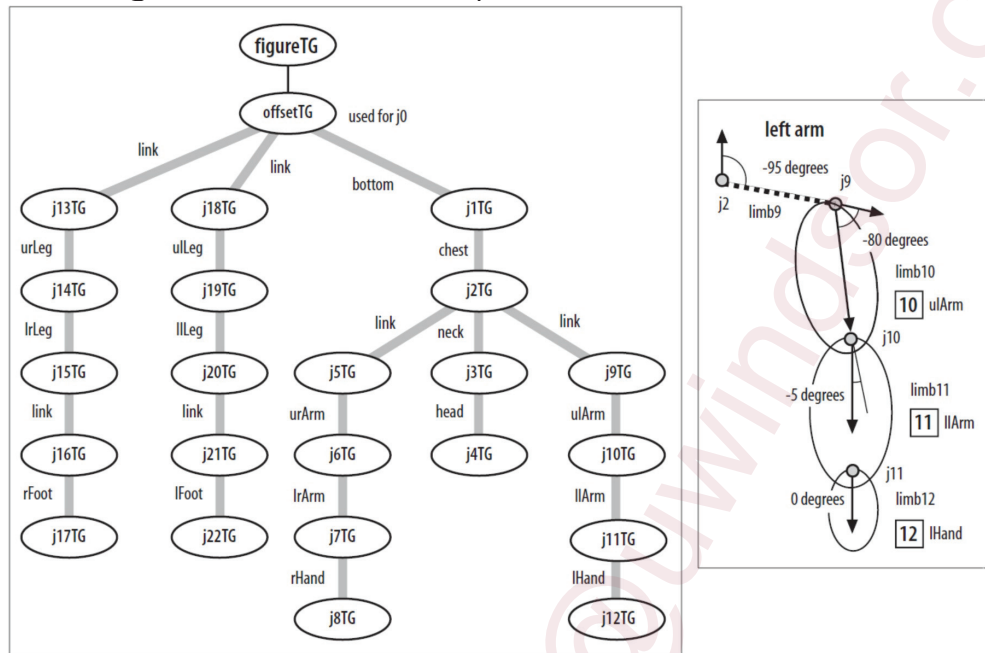
- Example: initial position and after limb movement



2. The figure's (named and numbered) limbs and joints



3. The figure's TransformGroups and left arm in detail

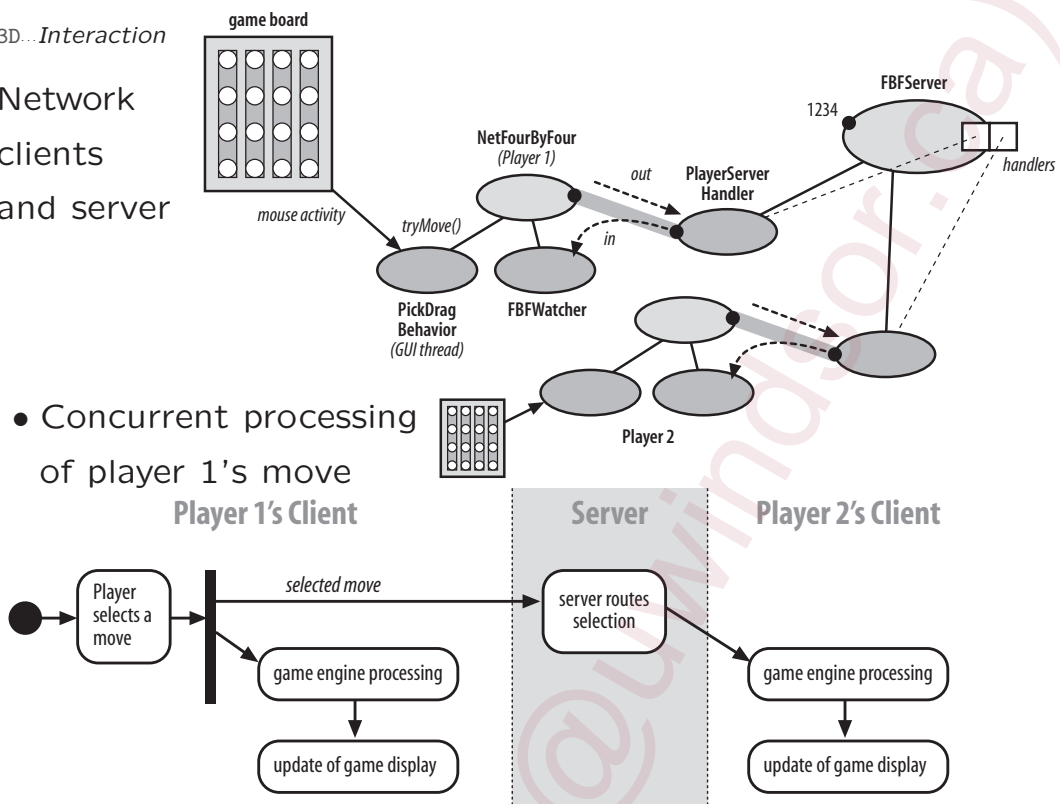


• Controlling the ranges

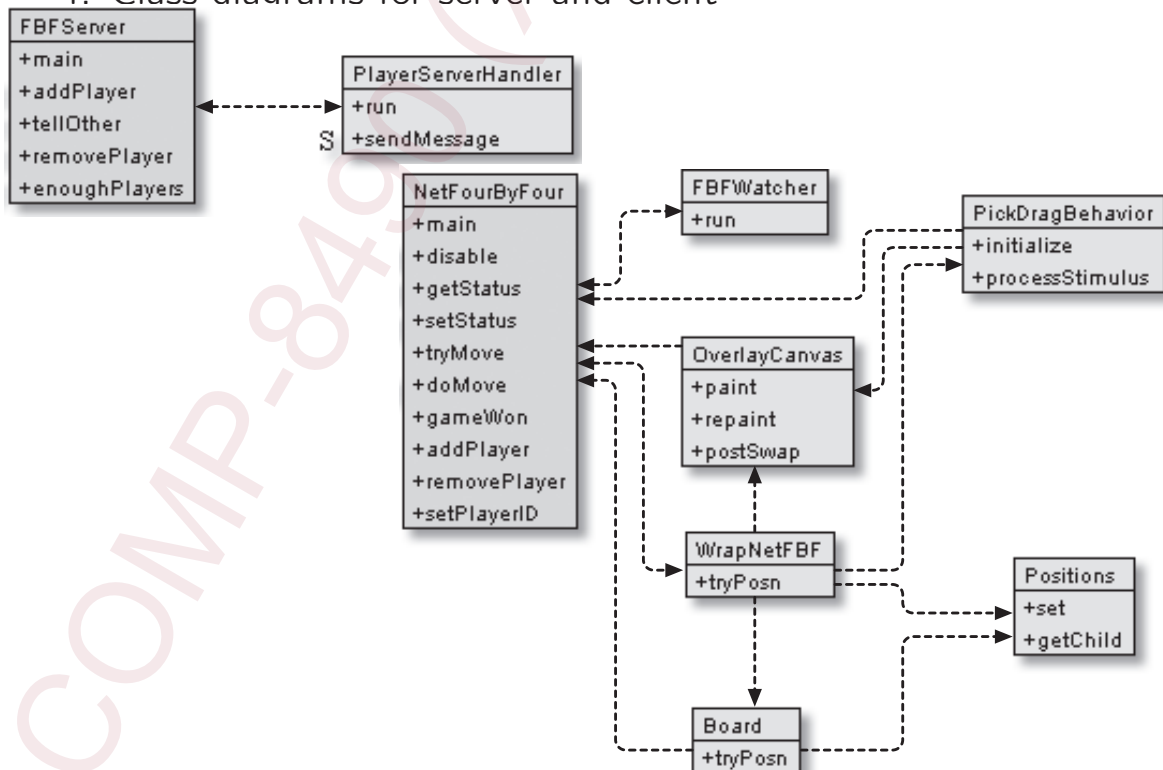
```
public void setRanges(double x1, double x2, double y1, double y2,
                     double z1, double z2) {
    setRange(X_AXIS, x1, x2);           // set all the axis ranges
    setRange(Y_AXIS, y1, y2);
    setRange(Z_AXIS, z1, z2);
} // end of setRanges()

public void setRange(int axis, double angle1, double angle2) {
    // set the range for axis only
    if (angle1 > angle2) {
        System.out.println(limbName + ": range in wrong order... swapping");
        double temp = angle1;
        angle1 = angle2;
        angle2 = temp;
    }
    if (axis == X_AXIS) {
        xMin = angle1; xMax = angle2; }
    else if (axis == Y_AXIS) {
        yMin = angle1; yMax = angle2; }
    else {
        zMin = angle1; zMax = angle2; } // Z_AXIS
} // end of setRange()
```

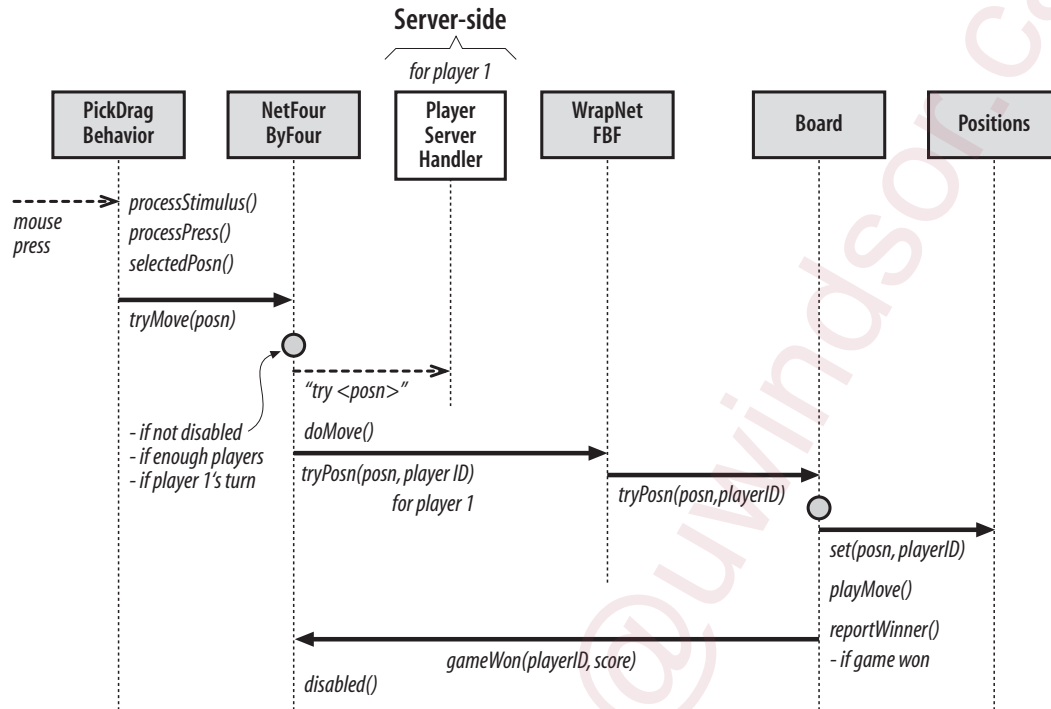

3. Network clients and server



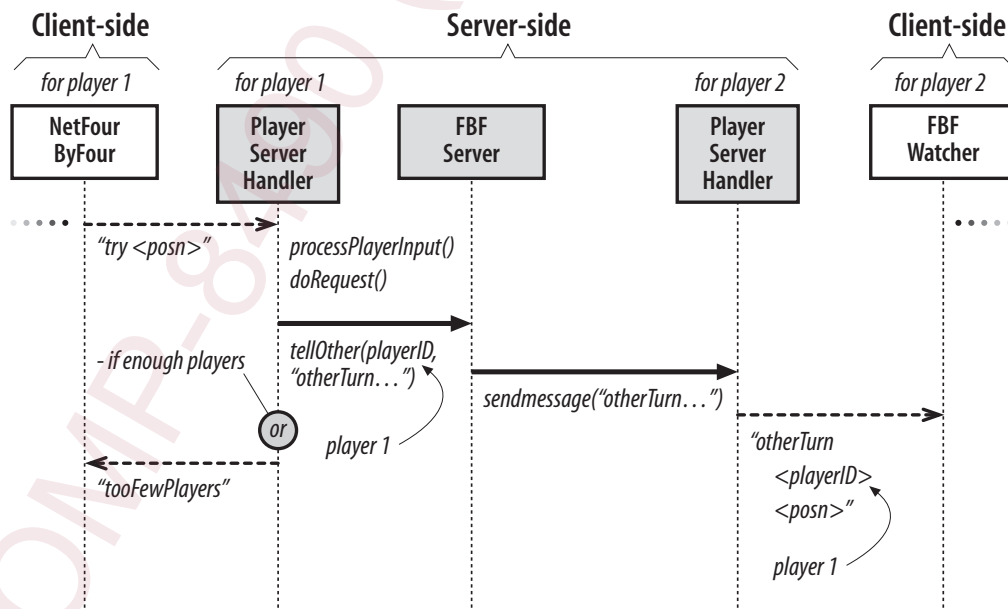
4. Class diagrams for server and client



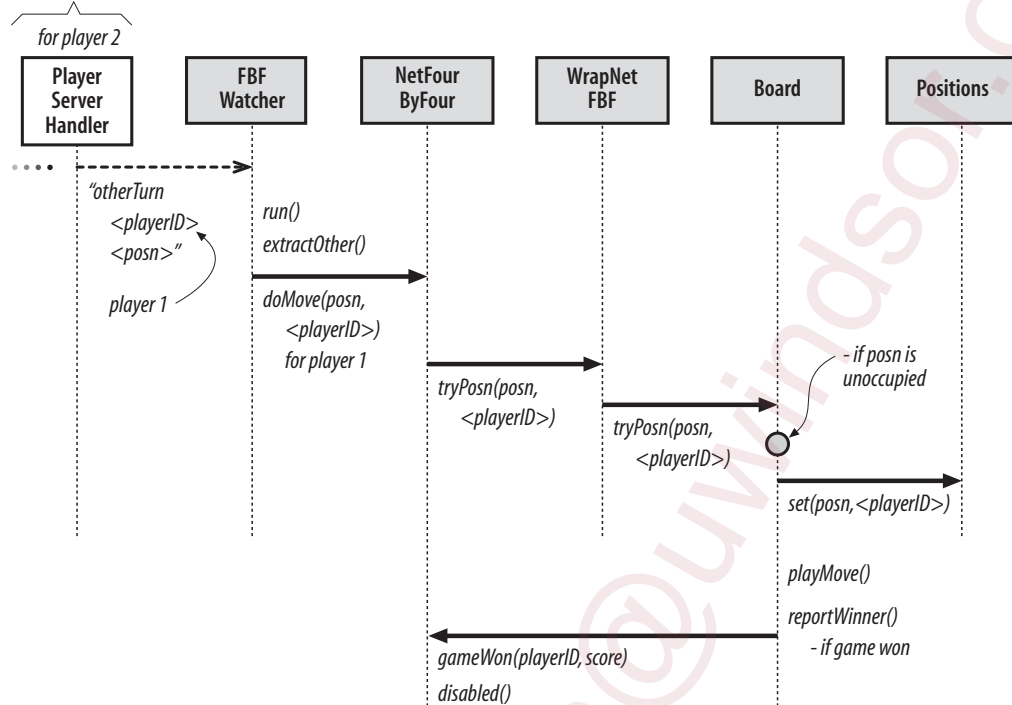
- Sequence of actions: player 1's side



- Sequence of actions: at the server



- Sequence of actions: player 2's side
Server-side



PART VI. PERFORMANCE IMPROVEMENT

1. Performance Considerations:

- Use of textures
 - Use low polygon geometry with high quality textures.
 - Use high quality textures with transparency and 2D geometry.
 - Use tiling and small texture sizes when possible.
- When creating multiple instances, use SharedGroup and Link nodes.
- Use Behavior and Alpha nodes only where necessary.
- Turn Timer nodes on and off where appropriate.

2. Examine every “new” statement used for object creations

- Replace a “new” statement with a static node if it is executed at every timer or render time.
- Try to create all nodes before and avoid using the “new” statement after a user starts interacting with your program.
- For garbage collection to take place use class `WeakHashMap`
 - The use of a `WeakHashMap` object associates pairs of objects as keys and values. The value is made eligible for garbage collection when the key becomes weakly reachable.
- For garbage collection to take place, unregister event listener classes when they are no longer needed.

3. There are a number of things in the Java 3D API that were included specifically to increase performance.

- *Capability bits* are the applications way of describing its intentions to the Java 3D implementation. The implementation examines the capability bits to determine which objects may change at run time. Many optimizations are possible with this feature.
- The default `isFrequent` bit indicates that the application may frequently access or modify those attributes permitted by the associated capability bit. This can be used by Java 3D as a hint to avoid certain optimizations that could cause those accesses or modifications to be expensive.

- There are two compile methods in Java 3D. They are in the `BranchGroup` and `SharedGroup` classes. Once an application calls `compile()`, only those attributes of objects that have their capability bits set may be modified. The implementation may then use this information to “compile” the data into a more efficient rendering format.
- Many Java 3D objects require a *bounds* associated with them. These objects include `Lights`, `Behaviors`, `Fogs`, `Clips`, `Backgrounds`, `BoundingLeafs`, `Sounds`, `Soundscapes`, `ModelClips`, and `AlternateAppearance`. These bounds limit the spatial scope of the specific object, allowing the implementation to quickly disregard the processing of any objects that are out of the spatial scope.

- All state required to render a specific object in Java 3D is completely defined by the direct path from the root node to the given leaf. That means that leaf nodes have no effect on other leaf nodes, and therefore may be rendered in any order. There are a few ordering requirements for direct descendants of `OrderedGroup` nodes or `Transparent` objects. But, most leaf nodes may be reordered to facilitate more efficient rendering.
- `OrderedGroup` supports an indirection table to allow the user to specify the order that the children should be rendered. This will speed up order update processing, eliminating the expensive attach and detach cycle.

- A `Shape3D` node has a reference to a `Geometry` and an `Appearance`. An `Appearance NodeComponent` is simply a collection of other `NodeComponent` references that describe the rendering characteristics of the geometry. Because the `Appearance` is nothing but a collection of references, it is much simpler and more efficient for the implementation to check for rendering characteristic changes when rendering. This allows the implementation to minimize state changes in the low level rendering API.

4. Tips and Tricks

- Move Object vs. Move `ViewPlatform`
 - If the application simply needs to transform the entire scene, transform the `ViewPlatform` instead. This changes the problem from transforming every object in the scene into only transforming the `ViewPlatform`.
- Capability bits
 - Only set them when needed. Many optimizations can be done when they are not set. So, plan out application requirements and only set the capability bits that are needed.

- **Bounds and Activation Radius**
 - Consider the spatial extent of various leaf nodes in the scene and assign bounds accordingly. This allows the implementation to prune processing on objects that are not in close proximity.
 - Automatic bounds calculations for geometric objects is fine.
 - Bounds computation does consume CPU cycles. If an application does a lot of geometry coordinate updates, to improve performance, it is better to turn off auto bounds compute. The application will have to do the bounds update itself.

- **Change Number of Shape3D Nodes**
 - In the current implementation there is a certain amount of fixed overhead associated with the use of the Shape3D node. In general, the fewer Shape3D nodes that an application uses, the better. However, combining Shape3D nodes without factoring in the spatial locality of the nodes to be combined can adversely effect performance by effectively disabling view frustum culling. An application programmer will need to experiment to find the right balance of combining Shape3D nodes while leveraging view frustum culling. The .compile optimization that combines shape node will do this automatically, when possible.

- Geometry Type and Format
 - Most rendering hardware reaches peak performance when rendering long triangle strips. Unfortunately, most geometry data stored in files is organized as independent triangles or small triangle fans (polygons). The Java 3D utility package includes a stripifier utility that will try to convert a given geometry type into long triangle strips. Application programmers should experiment with the stripifier to see if it helps with their specific data. If not, any stripification that the application can do will help.
 - Another option is that most rendering hardware can process a long list of independent triangles faster than

a long list of single triangle triangle fans. The stripifier in the Java 3D utility package will be continually updated to provide better stripification.

- Sharing Appearance/Texture/Material NodeComponents
 - To assist the implementation in efficient state sorting, and allow more shape nodes to be combined during compilation, applications can help by sharing Appearance/Texture/Material NodeComponent objects when possible.
- Geometry by reference
 - Using geometry by reference reduces the memory needed to store a scene graph, since Java 3D avoids creating a copy in some cases.

- However, using this features prevents Java 3D from creating display lists (unless the scene graph is compiled), so rendering performance can suffer in some cases. It is appropriate if memory is a concern or if the geometry is writable and may change frequently. The interleaved format will perform better than the non-interleaved formats, and should be used where possible. In by-reference mode, an application should use arrays of native data types; referring to *TupleXX[]* arrays should be avoided.
- Texture by reference and Y-up
 - Using texture by reference and Y-up format may reduce the memory needed to store a texture object,

since Java 3D avoids creating a copy in some cases. When a copy of the by-reference data is made in Java3D, users should be aware that this case will use twice as much memory as the by copy case. This is due to the fact that Java3D internally makes a copy in addition to the user's copy to the reference data.

- Drawing 2D graphics using J3DGraphics2D
 - The J3DGraphics2D class allows you to mix 2D and 3D drawing into the same window. However, this can be very slow in many cases because Java 3D needs to buffer up all of the data and then composite it into the back buffer of the Canvas3D.
 - A new method, `drawAndFlushImage`, is provided to ac-

celerate the drawing of 2D images into a Canvas3D. To use this, it is recommended that an application create their own `BufferedImage` of the desired size, use Java2D to render into their `BufferedImage`, and then use the new `drawAndFlushImage` method to draw the image into the Canvas3D.

- Application Threads

- The built in threads support in the Java language is very powerful, but can be deadly to performance if it is not controlled. Applications need to be very careful in their threads usage.
 - First, try to use them in a demand driven fashion. Only let the thread run when it has a task to do.

Free running threads can take a lot of cpu cycles from the rest of the threads in the system — including Java 3D threads.

- Next, be sure the priority of the threads are appropriate. Most Java Virtual Machines will enforce priorities aggressively. Too low a priority will starve the thread and too high a priority will starve the rest of the system. If in doubt, use the default thread priority.
- Finally, see if the application thread really needs to be a thread. Would the task that the thread performs be all right if it only ran once per frame? If so, consider changing the task to a Behavior that

wakes up each frame.

- Java 3D Threads
 - Java 3D uses many threads in its implementation, so it also needs to implement the precautions listed above. In almost all cases, Java 3D manages its threads efficiently. They are demand driven with default priorities. There are a few cases that don't follow these guidelines completely.
- Switch Nodes for Occlusion Culling
 - If the application is a first person point of view application, and the environment is well known, `Switch` nodes may be used to implement simple occlusion culling. The children of the switch node that are not

currently visible may be turned off. If the application has this kind of knowledge, this can be a very useful technique.

- Switch Nodes for Animation
 - Most animation is accomplished by changing the transformations that effect an object. If the animation is fairly simple and repeatable, the flip-book trick can be used to display the animation. Simply put all the animation frames under one switch node and use a `SwitchValueInterpolator` on the switch node. This increases memory consumption in favor of smooth animations.
- `OrderedGroup` Nodes

- `OrderedGroup` and its subclasses are not as high performing as the unordered group nodes. They disable any state sorting optimizations that are possible. If the application can find alternative solutions, performance will improve.
- **LOD Behaviors**
 - For complex scenes, using LOD Behaviors can improve performance by reducing geometry needed to render objects that don't need high level of detail. This is another option that increases memory consumption for faster render rates.
- **Picking**
 - If the application doesn't need the accuracy of ge-

ometry based picking, use bounds based picking. For more accurate picking and better picking performance, use `PickRay` instead of `PickCone/PickCylinder` unless you need to pick line/point. `PickCanvas` with a tolerance of 0 will use `PickRay` for picking.

- **D3D users only**
 - Using Quad with Polygon line mode is very slow. This is because DirectX doesn't support Quad. Breaking down the Quad into two triangles causes the diagonal line to be displayed. Instead Java 3D draws the polygon line and does the hidden surface removal manually.